Your Channel on Autopilot

How a free robot makes and posts a YouTube video,
a Short and an Instagram Reel every morning at 8:00 AM.



YouTube video · YouTube Short · Instagram Reel · 100% free tools

Written for people who have never seen a line of code.
Pictures first, simple words always. Technical details are at the end.

Project name: autotube

What is inside

What is inside . . . 2
1 The idea in one minute . . . 3
2 The big picture . . . 4
3 Meet the team . . . 5
4 A day in the life . . . 6
5 Inside one video . . . 7
6 Topics and playlists . . . 9
7 Thumbnails . . . 10
8 How the Instagram part works . . . 12
9 Keys and safety . . . 13
10 One-time setup recap . . . 14
11 Day-to-day care . . . 15
12 If something goes wrong . . . 16
13 Questions people ask . . . 17
14 Glossary . . . 18
15 Explain it to anyone . . . 19
16 Project facts at a glance . . . 20
+ Appendix: every file, line by line . . . 21
The repository map . . . 21
history.txt: the memory file . . . 21
requirements.txt . . . 22
.github/workflows/daily.yml . . . 22
get_token.py (runs once on your PC) . . . 24
get_ig.py (runs once on your PC) . . . 24
main.py, line by line . . . 26

How to read this guide

Sections 1 to 8 explain the idea with pictures and need no technical knowledge. Sections 9 to 15 are for the owner: safety, setup, daily care and fixing problems. The Appendix explains every file and every line of code in plain words.

1 The idea in one minute

Imagine you hired a very hard-working assistant. Every morning, before you wake up, the assistant picks a topic, writes a script, records a voice, finds video clips, edits them into a video, designs a cover picture, and posts everything on YouTube and Instagram at exactly **8:00 AM**.

This project is that assistant, except it is not a person. It is a small computer program of about 500 lines that lives on **GitHub**, a free website where programmers keep programs and run them. After the one-time setup, **your own computer and phone are not needed**.

> **In one sentence**
>
> Every day at 8:00 AM, a free program on GitHub publishes a new educational video, a Short and an Instagram Reel, completely on its own.

What you get every day

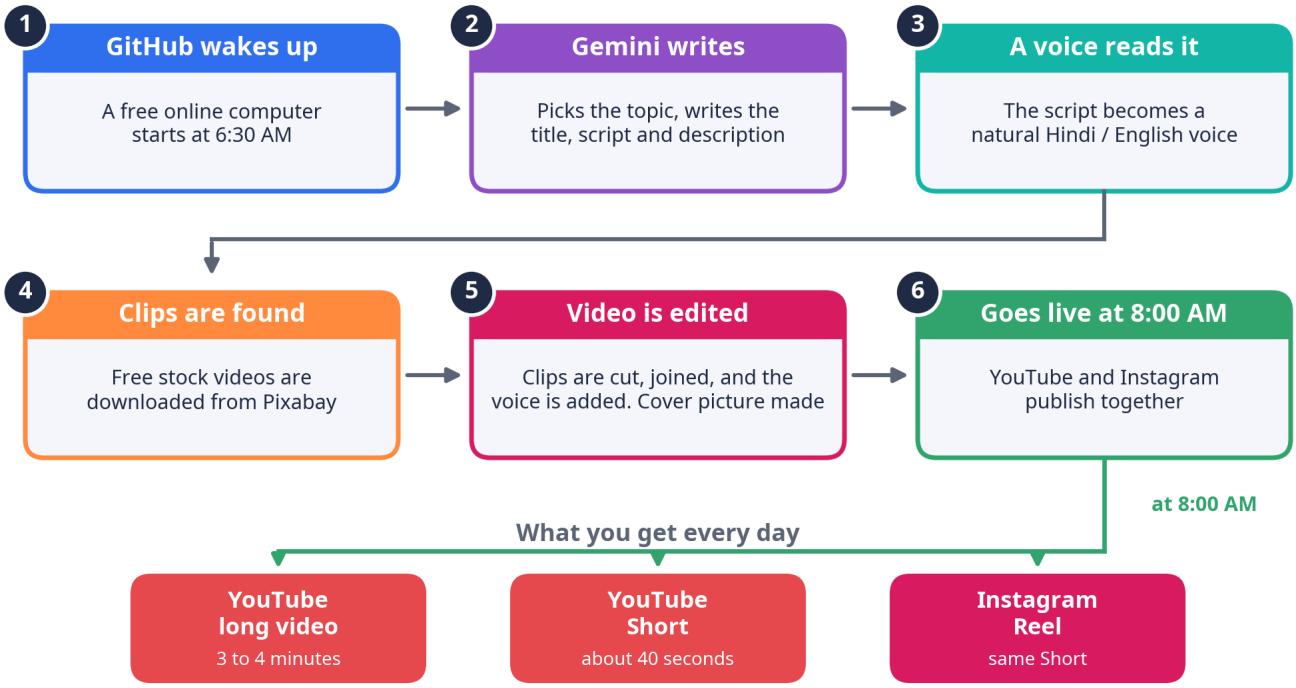
Item	Details
Long video	3 to 4 minutes, Full HD, with title, description, tags, a custom thumbnail, and a place in a topic playlist.
Short	About 40 seconds, vertical for phones, marked #Shorts, placed in the Shorts playlist.
Instagram Reel	The same Short, posted on Instagram with a caption.
Topic	One of 10 topics (cars, rockets, aeroplanes, AI and more). It rotates, and a title is never repeated.
Language	Hindi or English, chosen once with a single setting.

What it costs

Part	Cost
GitHub Actions (the computer that does the work)	Free for public repositories
Gemini (the AI writer)	Free tier
Pixabay (video clips)	Free
edge-tts (the voice)	Free
YouTube and Instagram connections (APIs)	Free, with daily limits
Total	**Rs 0**. Free limits can change, so look at them once in a while.

2 The big picture

The big picture: what the robot does every morning



Here is each step in simple words.

1 GitHub wakes up
GitHub has a built-in alarm clock called **Actions**. At 6:30 AM India time it starts a fresh, temporary computer and tells it to run the program called `main.py`.

2 Gemini writes
The program sends instructions to **Gemini**, Google's AI: 'You are an educational YouTube creator. Today's topic is Space. Give me a title, a script, a description and tags.' Gemini answers with everything in a tidy list. The program checks the answer carefully and asks again if it is bad.

3 A voice reads it
edge-tts, a free voice service, reads the script aloud in Hindi or English and saves it as an audio file.

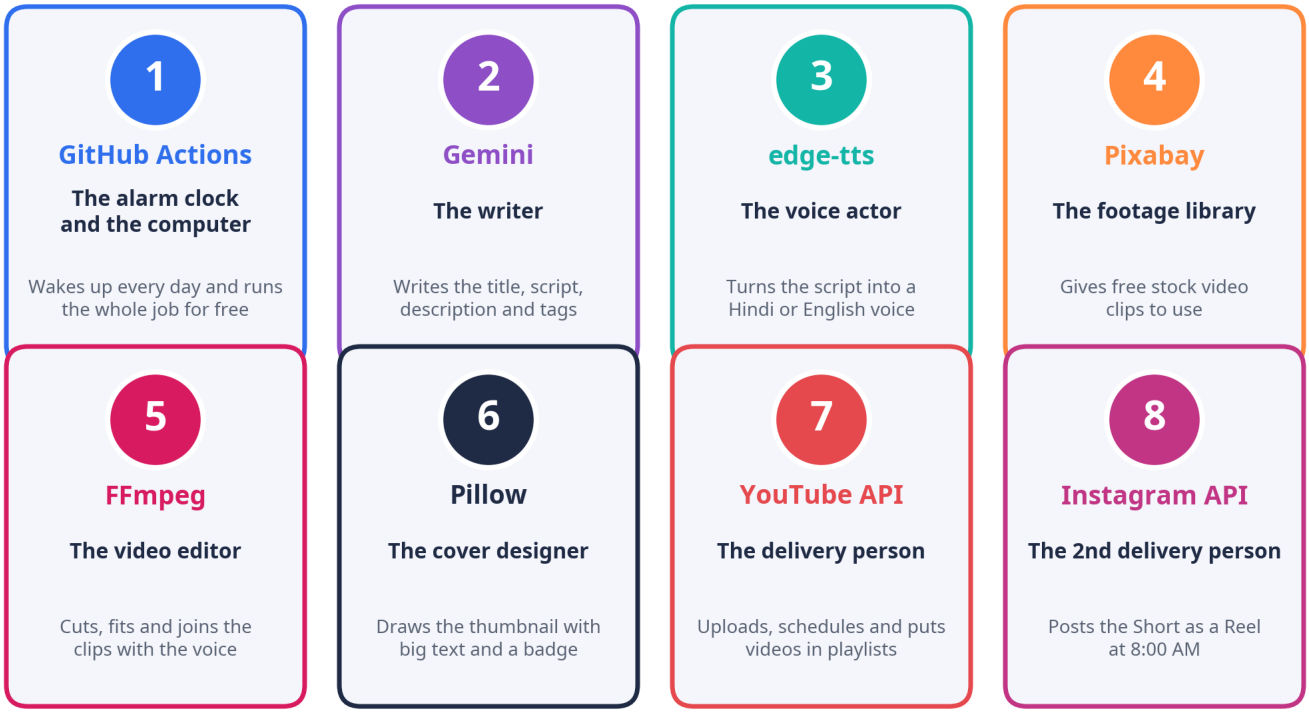
4 Clips are found
For each search word (for example 'rocket launch') the program asks **Pixabay**, a free video library, and downloads a clip.

5 Video is edited
FFmpeg, a free video editor that works with typed commands, cuts the clips, joins them and lays the voice on top. **Pillow** draws the thumbnail.

6 Goes live at 8:00 AM
The program uploads to YouTube with a timer set for 8:00. The Short also goes to Instagram. At 8:00 everything appears.

3 Meet the team

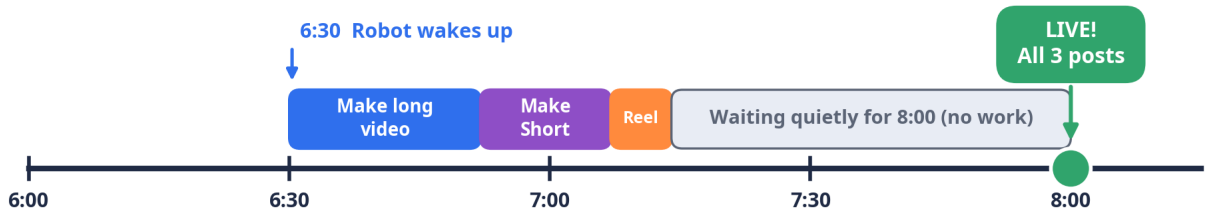
Meet the team: 8 helpers, all free



Helper	Its job in simple words	Real-life comparison	Website
GitHub Actions	Runs the whole job every day on a free computer	An alarm clock plus a kitchen	github.com
Gemini	Writes title, script, description, tags and search words	A writer	aistudio.google.com
edge-tts	Turns text into speech (Hindi or English)	A voice actor	built into the program
Pixabay	Gives free stock video clips	A footage library	pixabay.com
FFmpeg	Cuts and joins clips, adds the voice	A video editor	ffmpeg.org
Pillow	Draws the thumbnail with text	A cover designer	built into the program
YouTube API	Uploads, schedules, adds to playlists	A delivery person	console.cloud.google.com
Instagram API	Posts the Short as a Reel	A second delivery person	developers.facebook.com

4 A day in the life

A day in the life of the robot (Indian Standard Time)



YouTube keeps the two videos hidden and 'scheduled', like a timed email.
Instagram's Reel is prepared early and published by the robot at 8:00.

Times are approximate. Your phone and computer can be switched off the whole time.

Why start at 6:30 and not at 8:00? Making a video takes 15 to 30 minutes, and GitHub sometimes starts a few minutes late. So the robot starts early and finishes everything before 8:00.

How does it still go live at exactly 8:00? YouTube lets you upload a video as 'private' and choose the date and time it turns public. It works like scheduling an email. Both videos are uploaded early, wait hidden, and flip to public at 8:00.

And Instagram? Instagram's tool has no timer for this kind of post. So the robot uploads and prepares the Reel early, waits quietly, and presses 'publish' at 8:00.

What if it is very late? If the robot finishes after about 7:56 AM, it skips the waiting and publishes at once, so no day is lost. The same happens when you press the 'Run workflow' button yourself in the daytime.

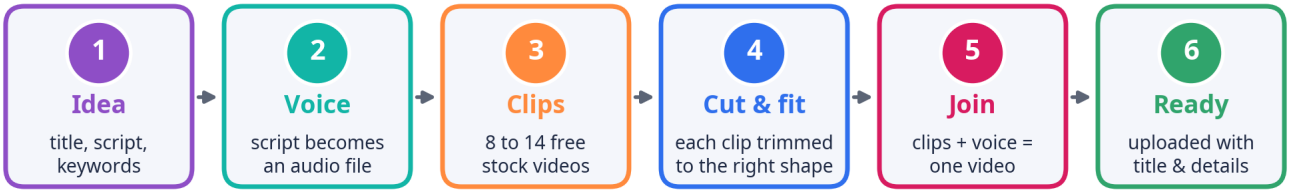
Time zones made easy

Local time (India)	World time (UTC)	What happens
6:30 AM	01:00	The robot wakes up (the cron setting in daily.yml)
8:00 AM	02:30	Everything goes live (the publish time in main.py)

Computers like to use UTC, a world clock without daylight saving. India is always UTC plus 5 hours 30 minutes.

5 Inside one video

Inside one video: how the pieces are made



Long video = wide screen (16:9). Short = tall phone screen (9:16).

1 Idea
Gemini returns the title, the description, 10 tags, the spoken script, a list of search words in the order of the script, and 2 to 4 words for the thumbnail.

2 Voice
edge-tts turns the script into an audio file. A long video uses normal speed. If a Short would be longer than 58 seconds, the voice is sped up a little (up to 30 percent).

3 Clips
14 clips for a long video, 6 for a Short. For every search word the robot picks randomly among the 5 best results. If a word finds nothing, it uses backup words for that topic. At least 3 clips are needed.

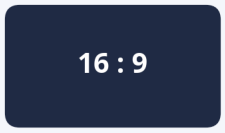
4 Cut and fit
Each clip is zoomed and trimmed to fill the screen shape: wide for long videos, tall for Shorts. A clip that is too short repeats itself.

5 Join
The pieces are joined end to end and the voice is added. The video is as long as the voice.

6 Ready
The video is uploaded with its title, description and tags, and (for long videos) the thumbnail.

Two kinds of video, made every day

LONG VIDEO



wide screen

- 3 to 4 minutes long
- Full HD (1920 x 1080)
- Has a custom thumbnail
- Goes into its topic playlist
- Posted on YouTube only

Best for learning something properly

SHORT / REEL



tall phone screen

- about 40 seconds long
- Full HD (1080 x 1920)
- YouTube picks a frame as cover
- Goes into 'Daily Shorts'
- YouTube Short + Instagram Reel

Best for quick discovery on phones

Both go live at 8:00 AM every day.

	Long video	Short / Reel
Length	3 to 4 minutes	About 40 seconds (never more than 58)
Shape	Wide, 16:9, 1920 x 1080	Tall, 9:16, 1080 x 1920
Script	450 to 550 words	90 to 110 words
Clips used	14	6 (bigger source files, so tall pictures stay sharp)
Cover picture	Custom thumbnail made by the robot	YouTube picks a frame
Playlist	Its topic playlist	Daily Shorts
Posted on	YouTube	YouTube and Instagram
Topic	Day 1 starts with Cars	Starts from the middle of the list, so it differs from the long video
Best at	Teaching one idea properly	Catching new viewers quickly

6 Topics and playlists

10 topics rotate, so the channel never gets boring



Long videos go in order (1, 2, 3 ...). Shorts start from the middle (6, 7, 8 ...), so the two videos of one day are never about the same topic.

Playlists

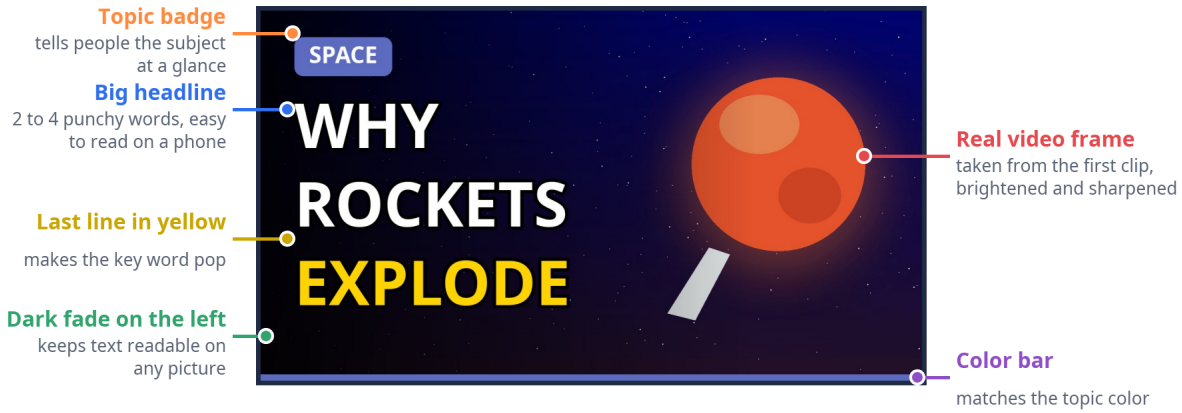
Video type	Which playlist	How it is made
Long video	Its topic playlist, for example 'Space & Rockets'	The first time a topic is used, the robot creates the playlist. After that it reuses it.
Short	One playlist called 'Daily Shorts' (in Hindi mode: the Hindi word for Shorts)	Created the first time, then reused.

It never repeats itself

Every title is saved in a memory file called `history.txt`. The next day the robot shows the AI the last 60 titles and says 'never repeat or copy these'. The same file also tells the robot which topic is next.

7 Thumbnails

Anatomy of a thumbnail (the cover picture)



Only the long video gets this. YouTube picks a frame for Shorts by itself.

A thumbnail is the small cover picture people see before they decide to click. The robot makes a new one for every long video.

1. **Take a frame**
   It grabs one still picture, 1 second into the first clip.
2. **Improve the colors**
   Colors get 25 percent stronger, contrast 10 percent stronger, and the picture is made slightly darker.
3. **Add a dark fade**
   A black-to-clear fade on the left side makes the text readable on any picture.
4. **Fit the headline**
   The text starts huge and is shrunk step by step until it fits in at most 3 lines.
5. **Draw the text**
   White letters with a black outline. The last line is yellow so the key word pops.
6. **Add the badge and bar**
   A colored badge with the topic name at the top and a color bar at the bottom. Each topic has its own color.
7. **Save small**
   Saved as a JPEG under 2 MB, which is YouTube's limit.

Hindi and English together

A Hindi font cannot draw English letters, and an English font cannot draw Hindi letters. So the program cuts the text into pieces and draws every piece with the right font. Words like 'AI' next to Hindi words work fine.

8 How the Instagram part works

Only the Short goes to Instagram, as a Reel. The posting has four small steps:

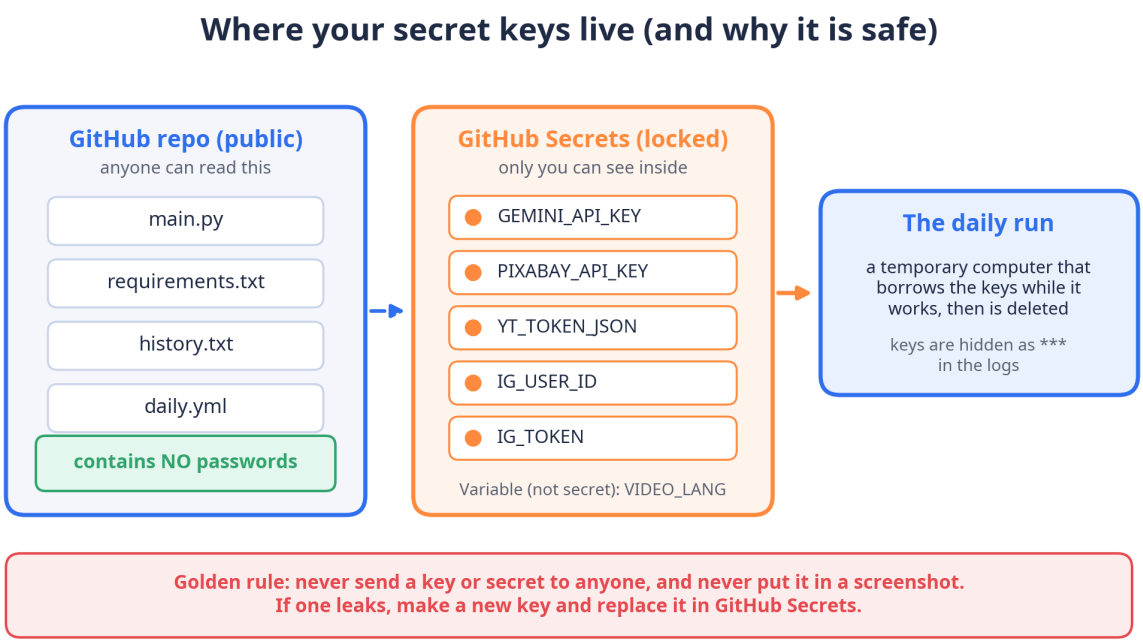
1. **Create a draft**
   The robot tells Instagram: 'I am about to upload a Reel' and sends the caption (title plus description).
2. **Upload the file**
   The video file is sent to the address Instagram gives back.
3. **Wait for processing**
   Instagram checks the video. The robot asks every 10 seconds until the status says FINISHED.
4. **Publish at 8:00**
   The robot waits until 8:00 AM and then presses publish. The Reel appears on your profile.

Why is an empty Facebook Page needed?

Instagram's posting tool is built by Meta, and it officially works through a Facebook Page that is linked to the Instagram account. The Page stays empty and can be set to 'Unpublished'. **The robot never posts anything on Facebook.**

What is needed	Why
Instagram Professional (Creator) account	Normal personal accounts cannot be controlled by programs.
An empty Facebook Page linked to it	The bridge Instagram's tool uses.
A Meta developer app (autotube-ig) with 5 permissions	The official permission to post.
A key that never expires (made by get_ig.py)	So nothing has to be renewed every 60 days.

9 Keys and safety



A **key** is like a password that lets a program use a service on your behalf. Keys must stay private.

Secret name	Comes from	What it allows
GEMINI_API_KEY	Google AI Studio	Using the AI writer
PIXABAY_API_KEY	Pixabay	Downloading video clips
YT_TOKEN_JSON	Google login (get_token.py)	Uploading to your YouTube channel and making playlists
IG_USER_ID	get_ig.py	Says which Instagram account to post to (not secret by itself)
IG_TOKEN	get_ig.py	Posting to your Instagram account

Rules

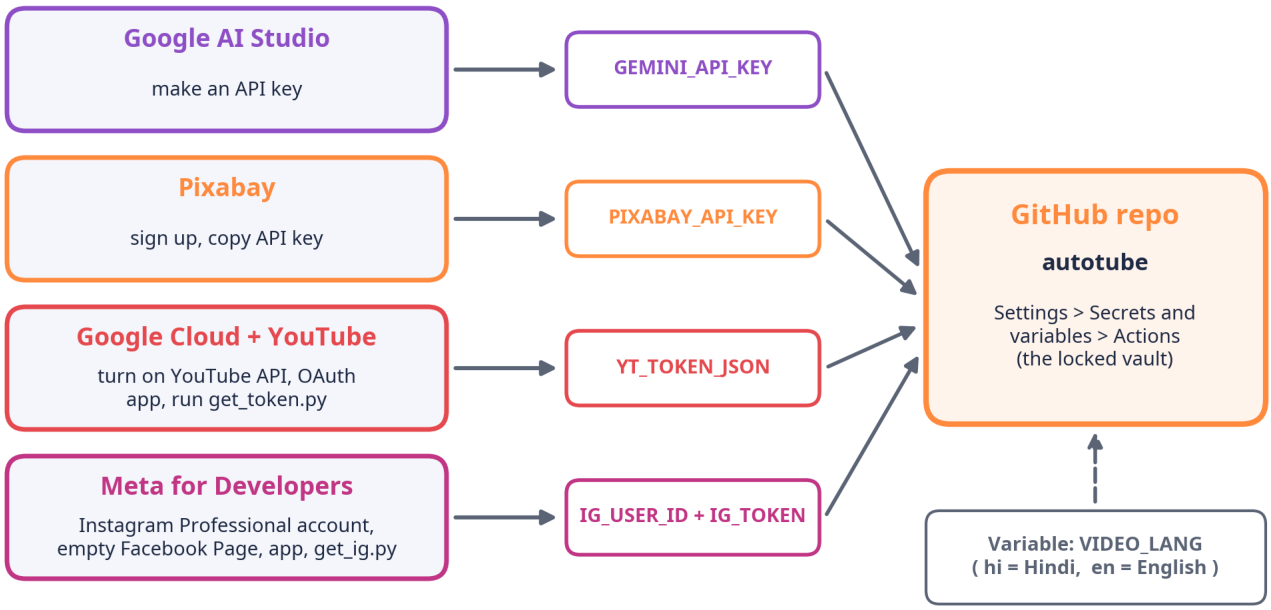
- Never send a key to anyone, never post it, never put it in a screenshot.
- Keys belong only in GitHub, under **Settings, Secrets and variables, Actions**. Never write them inside the code.
- The files client_secret.json, token.json and ig_keys.txt on your computer are private. Do not upload them to GitHub.
- If a key leaks: make a new key at the same website and replace the secret in GitHub.

Is it safe that the repository is public?

Yes, because the repository holds only the program and the memory file, never the keys. A public repository also gets free running time. A private one has a limited number of free minutes per month, and the daily waiting makes each run long, so keep it **Public**.

10 One-time setup recap

One-time setup: where each key came from



Every key is created once and pasted into GitHub. After that, nothing is needed from you.

#	Where	What was done	Result
1	aistudio.google.com	Created an API key	GEMINI_API_KEY
2	pixabay.com/api/docs	Signed up and copied the API key	PIXABAY_API_KEY
3	console.cloud.google.com	New project, turned on YouTube Data API v3, set up the OAuth screen (Branding, published the app, added the YouTube permission), created a Desktop client	client_secret.json
4	Your PC	Installed Python, ran get_token.py, logged in and pressed Allow	token.json, copied into YT_TOKEN_JSON
5	Instagram app	Switched to a Professional (Creator) account	Ready for the API
6	facebook.com	Made an empty Page and linked it to Instagram	The bridge
7	developers.facebook.com	Made the app autotube-ig, added 5 permissions, made a temporary token in Graph API Explorer, ran get_ig.py	IG_USER_ID, IG_TOKEN
8	github.com	Made the repo, added 4 files, 5 secrets and 1 variable	The robot is ready
9	GitHub, Actions tab	Pressed 'Run workflow' as a test	A green tick and live videos

11 Day-to-day care

Daily

Nothing. The robot does everything. You do not need to open any website.

Once a week (5 minutes)

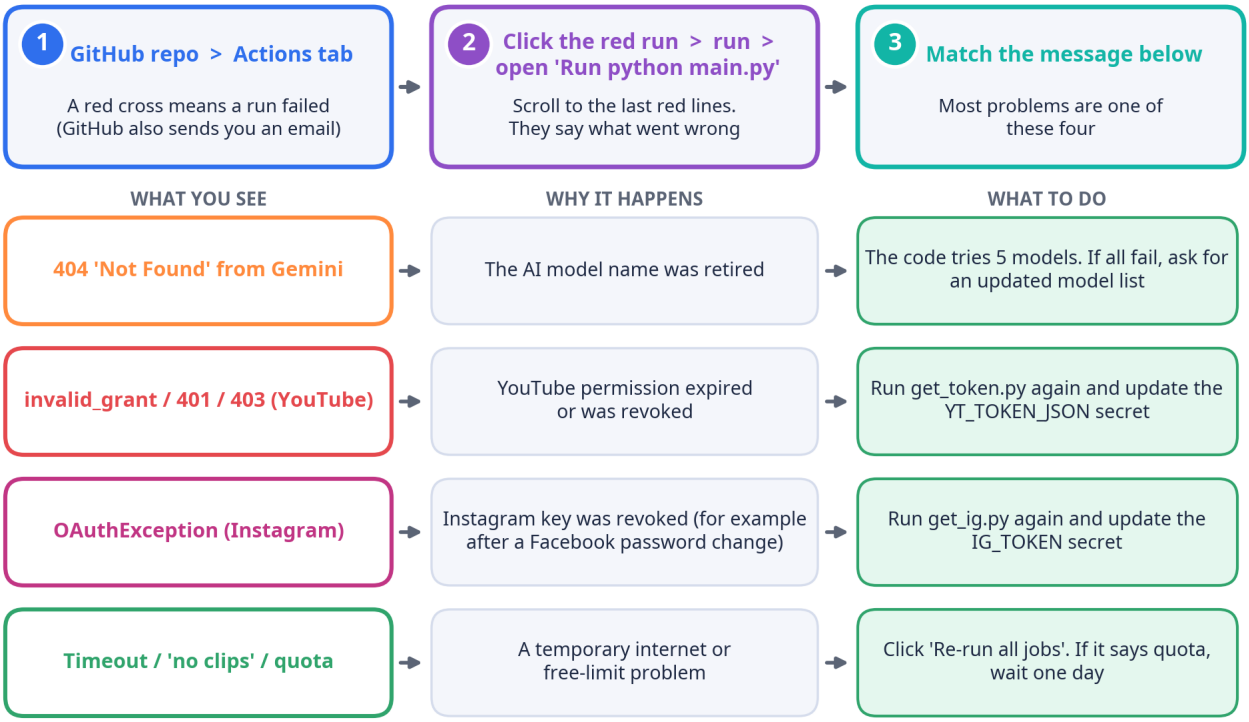
- Open YouTube Studio, then Content, and look at the last videos. Do they look good? Is anything marked in the Restrictions column?
- Watch one video and check the facts. AI can make mistakes. Delete a video if something is wrong.
- Open your email. YouTube writes to the Google account that owns the channel about copyright claims or guideline problems. Check Spam too.
- On GitHub, open the Actions tab. All runs should have a green tick.

What could stop the robot

What happens	Result	What to do
You change your Google password	The YouTube permission may stop working	Run get_token.py again and update the YT_TOKEN_JSON secret
You change your Facebook password or leave the Page	The Instagram key may stop working	Run get_ig.py again and update the IG_TOKEN secret
Google retires an AI model name	The AI step fails	The code tries 5 model names. If all fail, update the list in main.py
Free limits change or are used up	A run fails with the word quota	Wait a day, or ask for a smaller plan
GitHub updates its tools	A yellow warning appears (for example about Node.js)	Harmless. Update the version numbers in daily.yml when it becomes an error
The repository is quiet for 60 days	GitHub pauses scheduled runs	Does not happen here: the robot saves history.txt every day
YouTube treats the channel as repetitive automatic content	Monetization can be limited	Check the Studio messages. Adding your own voice or edits helps

12 If something goes wrong

If something goes wrong: the 3-step check



Problems we already met while building it

Message or sign	Reason	Fix
pip is not recognized	Python was not installed, or not added to PATH	Install Python and tick 'Add python.exe to PATH', then open a new cmd window. Or use the word py instead of python
Publish app button is grey	The Branding page was not complete	Fill app name, support email, links, authorized domain and developer email, then Save
Secret names can only contain...	A space or odd character in the Name box	Retype the name with no spaces
404 Not Found (Gemini)	The AI model name was retired	The code now tries 5 models in a row
Pexels says new keys are paused	Pexels stopped giving keys	Pixabay is used instead
Error validating client secret	The App Secret did not match the app	Copy App ID and App Secret again from App settings, Basic
Generate Access Token is grey	Text typed in the token box, or the wrong token type	Clear the box, refresh, choose Get User Access Token

13 Questions people ask

Q. Do I need to keep my computer or phone on?
No. The work happens on GitHub's computers. Your devices can be off or in another city.

Q. Do I need internet at home?
No. Internet is only needed when you want to look at the results.

Q. I am logged in on my mobile. Does that matter?
No. The robot does not use your phone or your browser. It uses the keys stored in GitHub.

Q. Is it really free?
Yes. Every tool has a free tier. The free limits are large enough for one long video and one Short per day.

Q. How do I pause it?
GitHub, Actions tab, click daily-video, press the three dots, choose Disable workflow. Enable it again the same way.

Q. How do I run it right now?
Actions tab, daily-video, Run workflow. In the daytime it publishes immediately.

Q. How do I switch Hindi and English?
Settings, Secrets and variables, Actions, the Variables tab, edit VIDEO_LANG and type hi or en.

Q. How do I change the time?
Two places hold the time: the cron line in daily.yml (6:30 AM start) and the 02:30 in the publish_time function of main.py (8:00 AM go-live). Change both.

Q. Can I add a new topic?
Yes. Add one more block to the CATS list in main.py with a name, badge words, a color and 3 backup search words.

Q. How do I delete a bad video?
YouTube Studio, Content, three dots, Delete forever. On Instagram open the Reel, tap the three dots, Delete.

Q. Will I be told about copyright or guideline problems?
YouTube emails the owner of the channel's Google account and also shows a note in Studio under Content, in the Restrictions column. Check Spam as well.

Q. Are the facts always right?
The AI is told to use only well-known facts, but it can still be wrong. Spot-check videos every week.

Q. How many uploads can I make per day?
YouTube's free allowance is about 6 uploads per day. This project uses 2.

Q. Can someone else copy this project?
Yes. They copy the repository, make their own keys and add them as secrets. Section 10 shows the way.

14 Glossary

Word	Meaning in simple words
API	A doorway that lets one program ask another program to do something, for example 'upload this video'.
API key	A secret password that proves who is asking at the doorway.
Token	A permission slip made after you press Allow. It says 'this program may act for this account'.
OAuth	The safe login method used by Google and Meta, where you press Allow instead of giving your password.
Scope	One specific permission, for example 'manage my YouTube channel'.
GitHub	A free website where programs are stored and can be run.
Repository (repo)	A folder on GitHub that holds a project's files.
GitHub Actions	GitHub's built-in machine that runs a program on a schedule.
Workflow	The to-do list written in daily.yml that Actions follows.
Cron	A short code for a schedule, such as '0 1 * * *' meaning every day at 01:00.
UTC / IST	UTC is the world clock. IST is Indian time, UTC plus 5:30.
Secret	A private value saved in GitHub that programs can use but people cannot read.
Variable	A normal setting saved in GitHub, like VIDEO_LANG.
Commit	Saving a change to a file in the repository.
Prompt	The instruction message sent to the AI.
JSON	A neat way to write a list of items as text so programs can read it.
Script	Two meanings: the words spoken in the video, or a small program file.
FFmpeg	A free tool that cuts, joins and converts video using typed commands.
Thumbnail	The cover picture of a video.
Short / Reel	A tall, short video made for phones: YouTube calls it a Short, Instagram calls it a Reel.
Playlist	A named list of videos on YouTube.
Draft (container)	Instagram's name for a video that is uploaded but not yet published.
Python	The programming language the project is written in.
Library	A ready-made toolbox of code that someone else wrote.

15 Explain it to anyone

The 30-second version

'I built a small robot. Every morning at 6:30 it writes a script with AI, records a voice, finds free video clips, edits a video and makes a cover picture. At exactly 8:00 it publishes a video and a Short on YouTube and a Reel on Instagram. It runs on a free website, so my computer can be off, and it costs nothing.'

The restaurant comparison (2 minutes)

In the project	In a restaurant
GitHub	The kitchen that opens at 6:30 every morning
Gemini	The person who writes today's recipe
Pixabay	The pantry full of free ingredients
edge-tts	The narrator who reads the menu aloud
FFmpeg	The chef who cooks everything together
Pillow	The designer of the menu cover
YouTube and Instagram	Two delivery services that bring the meal to customers at 8:00
Secrets	The keys to the kitchen, kept in a locked safe

Questions you may get

Question	Short answer
Is a person writing the videos?	An AI writes the words, a voice tool speaks them, and a program edits the video. Facts are checked by me once a week.
Does it cost money?	No. Everything uses free plans.
Can it break?	Sometimes a free service changes. If it does, GitHub sends me an email and I fix it.
Can I make one?	Yes. You need free accounts on GitHub, Google, Pixabay and Meta, and this guide.

A 2-minute demo

- Open the repository on GitHub and point to the Actions tab with a green tick.
- Open YouTube Studio and show the newest long video, the Short and the playlists.
- Open Instagram and show the Reel.
- Press 'Run workflow' only if you have 25 minutes and want to show it live.

16 Project facts at a glance

Item	Value
Repository name	autotube (public)
Files in the repository	main.py, requirements.txt, history.txt, .github/workflows/daily.yml
Files only on your computer	client_secret.json, token.json, get_token.py, get_ig.py, ig_keys.txt (keep private)
Start time	6:30 AM India time (01:00 UTC) every day
Go-live time	8:00 AM India time (02:30 UTC)
Language	Hindi or English, set by the VIDEO_LANG variable
Topics	10, rotating
Long video	3 to 4 minutes, 1920 x 1080, custom thumbnail, topic playlist
Short	About 40 seconds, 1080 x 1920, Daily Shorts playlist, also posted as an Instagram Reel
AI models tried (in order)	gemini-3.5-flash, gemini-3.1-flash-lite, gemini-3-flash-preview, gemini-2.5-flash-lite, gemini-2.5-flash
Voices	hi-IN-MadhurNeural (Hindi), en-US-GuyNeural (English)
Secrets	GEMINI_API_KEY, PIXABAY_API_KEY, YT_TOKEN_JSON, IG_USER_ID, IG_TOKEN
Variable	VIDEO_LANG
YouTube category	Education (not made for kids)
Running time of one run	About 25 minutes of work plus waiting until 8:00

+ Appendix: every file, line by line

This part is for the curious. For every file, each line of code is shown on the left and explained in plain words on the right. The line numbers match the real files. In this PDF, Hindi words inside the code are shown as `{hindi}` because the code font cannot draw them.

The repository map

File	What it is	Who changes it
main.py	The program: the brain of the robot (497 lines)	Only when you want to change how it works
requirements.txt	A shopping list of the Python tools the program needs	Rarely
.github/workflows/daily.yml	The alarm clock and the to-do list for GitHub	To change the time or settings
history.txt	The memory: titles and topics of past videos	The robot, every day
get_token.py (PC only)	One-time helper that makes the YouTube permission slip	Never, unless a token must be renewed
get_ig.py (PC only)	One-time helper that makes the Instagram keys	Never, unless a key must be renewed

history.txt: the memory file

Each line is one finished video, written as three parts separated by a vertical bar: the kind (long or short), the topic, and the title. For example:

```
long|Space & Rockets|Why Rockets Fly
short|Technology & Gadgets|How Phones Know Where You Are
```

The robot counts the lines of each kind to know which topic is next, and sends the titles to the AI so they are not repeated. The very first line 'start' is an old test line and is simply remembered as a used title.

requirements.txt

Code (line numbers on the left)	What it means
1 `pillow`	pillow: draws the thumbnail picture.
2 `edge-tts`	edge-tts: makes the voice.
3 `requests`	requests: visits websites and APIs.
4 `google-api-python-client`	google-api-python-client: talks to YouTube.
5 `google-auth`	google-auth: handles the Google permission slip.
6 `google-auth-oauthlib`	google-auth-oauthlib: handles the Google login step.

.github/workflows/daily.yml

This file is not Python. It is written in YAML, a simple format where the indentation shows what belongs to what.

Code (line numbers on the left)	What it means
1 `name: daily-video`	The workflow's name, shown in GitHub's Actions tab.
2 `on:`	'on' lists what starts the workflow.
3 `  schedule:` 4 `    - cron: "0 1 * * *"`	A schedule: cron '0 1 * * *' means minute 0, hour 1, every day, in UTC. That is 6:30 AM in India.
5 `  workflow_dispatch:`	workflow_dispatch adds the manual 'Run workflow' button.
6 `permissions:` 7 `  contents: write`	Gives the workflow permission to write to the repository (needed to save history.txt).
8 `jobs:` 9 `  run:`	'jobs' lists the work to do. This workflow has one job called 'run'.
10 `    runs-on: ubuntu-24.04`	Run it on a fresh Ubuntu 24.04 Linux computer.
11 `    timeout-minutes: 240`	Give up after 240 minutes (4 hours) at most.
12 `    steps:`	'steps' are the actions, done one after another.
13 `      - uses: actions/checkout@v4`	Step 1: download a copy of your repository onto that computer.
14 `      - uses: actions/setup-python@v5` 15 `        with:` 16 `          python-version: "3.11"`	Step 2: install Python version 3.11.
17 `      - run: sudo apt-get update && sudo apt-get install -y ffmpeg fonts-noto-core fonts-dejavu-core libraqm0`	Step 3: install ffmpeg (video editor), the fonts, and a text-layout helper.
18 `      - run: pip install -r requirements.txt`	Step 4: install the Python tools listed in requirements.txt.
19 `      - run: python main.py`	Step 5: run main.py, which does all the work.
20 `        env:`	'env' lists the settings handed to main.py.
21 `          GEMINI_API_KEY: ${{ secrets.GEMINI_API_KEY }}` 22 `          PIXABAY_API_KEY: ${{ secrets.PIXABAY_API_KEY }}` 23 `          YT_TOKEN_JSON: ${{ secrets.YT_TOKEN_JSON }}` 24 `          IG_USER_ID: ${{ secrets.IG_USER_ID }}` 25 `          IG_TOKEN: ${{ secrets.IG_TOKEN }}`	The 5 secret keys, taken from GitHub's secret vault.
26 `          VIDEO_LANG: ${{ vars.VIDEO_LANG }}`	The language setting (hi or en), taken from the repository variables.
27 `      - if: always()` 28 `        run: |`	Step 6: 'if: always()' means this step runs even if the previous step failed. 'run: |' starts a block of commands.

Code (line numbers on the left)	What it means
29 git config user.name "bot" 30 git config user.email " bot@users.noreply.github.com"	Tells git (the version tool) who is saving: a bot with a no-reply email.
31 git add history.txt	Selects history.txt to be saved.
32 git commit -m "update history" || true	Saves it with the note 'update history'. '|| true' means 'do not complain if there is nothing new'.
33 git push	Sends the saved history back to your GitHub repository.

get_token.py (runs once on your PC)

Code (line numbers on the left)	What it means
`from google_auth_oauthlib.flow import InstalledAppFlow`	Brings in the login helper that talks to Google's sign-in page.
`flow = InstalledAppFlow.from_client_secrets_file( "client_secret.json", ["https://www.googleapis.com/auth/youtube"])`	Loads your app's ID card (client_secret.json) and states the permission it wants: full management of your YouTube account.
`creds = flow.run_local_server(port=0)`	Opens your browser to log in and press Allow. port=0 means 'choose any free door on this computer'.
`open("token.json", "w").write(creds.to_json())`	Saves the permission slip (token) into the file token.json.
`print("token.json created")`	Prints a message so you know it worked.

get_ig.py (runs once on your PC)

Code (line numbers on the left)	What it means
`import requests`	Brings in requests, which visits websites and APIs.
`G = "https://graph.facebook.com"`	The web address of Meta's service.
`def get(path, **params):`	Defines 'get', a helper that asks Meta a question. path is the question, params are the details.
`r = requests.get(f"{G}/{path}", params=params, timeout=60)`	Sends the question and waits up to 60 seconds.
`if r.status_code != 200: print("ERROR:", r.text) input("Press Enter to close") raise SystemExit(1)`	If Meta did not answer 'OK' (code 200): print the error, wait for you to press Enter (so you can read it) and stop.
`return r.json()`	Otherwise return the answer as data.
`app_id = input("App ID: ").strip()` `app_secret = input("App Secret: ").strip()` `short = input("Token from Graph API Explorer: ").strip()`	Asks you to type or paste 3 things: App ID, App Secret and the temporary token. .strip() removes stray spaces.
`long_user = get("oauth/access_token", grant_type="fb_exchange_token", client_id=app_id, client_secret=app_secret, fb_exchange_token=short)["access_token"]`	Swaps your short-lived temporary token for a long-lived one by showing Meta your App ID and App Secret.
`pages = get("me/accounts", fields="name,access_token,instagram_business_account", access_token=long_user).get("data", [])`	Asks for the list of your Facebook Pages with their keys and any linked Instagram account.
`found = None`	'found' starts as nothing.
`for p in pages: ig = p.get("instagram_business_account") if ig: found = (p["name"], ig["id"], p["access_token"]) break`	Goes through the Pages and picks the first one that has an Instagram account linked, saving its name, Instagram ID and key.
`if not found: print("No Instagram account is linked to your Facebook Pages.") print("Pages seen:", [p["name"] for p in pages]) input("Press Enter to close") raise SystemExit(1)`	If no Page has Instagram linked, print a helpful message, wait for Enter and stop.
`name, ig_id, token = found`	Unpacks the three saved values: Page name, Instagram ID and token.
`info = get("debug_token", input_token=token, access_token=f"{app_id}|{app_secret}")["data"]`	Asks Meta to inspect the token.
`print("Page:", name)` `print("Token never expires:", info.get("expires_at", 0) == 0)`	Prints the Page name and whether the token never expires (True is what you want).

Code (line numbers on the left)	What it means
43 with open("ig_keys.txt", "w") as f: 44 f.write(f"IG_USER_ID={ig_id}\nIG_TOKEN={token}\n")	Writes IG_USER_ID and IG_TOKEN into the file ig_keys.txt.
45 print("Done. Open ig_keys.txt")	Prints that it is done.

+ main.py, line by line

Part A: Imports and settings

The toolboxes the program uses and the settings it reads.

Code (line numbers on the left)	What it means
`1 import os, sys, json, asyncio, time, random, subprocess, datetime, requests`	Brings in the toolboxes the program needs: os (reads the computer's settings), sys (stops the program), json (a standard way to write data as text), asyncio (needed by the voice maker), time (waiting), random (picking by chance), subprocess (runs other programs such as ffmpeg), datetime (dates and clock) and requests (visits websites and APIs).
`2 import edge_tts`	Brings in edge-tts, the tool that turns written text into a spoken voice.
`3 from PIL import Image, ImageDraw, ImageFont, ImageEnhance, ImageOps`	Brings in Pillow's drawing tools. They are used to make the thumbnail picture.
`4 from google.oauth2.credentials import Credentials`	Brings in Credentials, the 'permission slip' that proves to Google that this program may act for your channel.
`5 from googleapiclient.discovery import build`	Brings in build, which creates the connection to the YouTube service.
`6 from googleapiclient.http import MediaFileUpload`	Brings in MediaFileUpload, which lets the program send a video or image file to YouTube.
`8 HI = os.environ.get("VIDEO_LANG", "hi").strip().lower() == "hi"`	HI becomes True when the VIDEO_LANG setting is 'hi' (Hindi). It reads the setting, removes stray spaces, makes it lowercase and compares it with 'hi'. If the setting is missing it assumes Hindi.
`9 GEMINI = os.environ.get("GEMINI_API_KEY", "")` `10 PIXABAY = os.environ.get("PIXABAY_API_KEY", "")`	Reads the Gemini key and the Pixabay key from GitHub's secret vault. If a key is missing it uses empty text.
`11 VOICE = "hi-IN-MadhurNeural" if HI else "en-US-GuyNeural"`	Chooses the voice: a Hindi male voice (Madhur) for Hindi, otherwise an English male voice (Guy).
`12 LANG_NAME = "Hindi (Devanagari script)" if HI else "English"`	The language name that is written into the instructions for the AI writer.
`13 MODELS = ["gemini-3.5-flash", "gemini-3.1-flash-lite", "gemini-3-flash-preview",` `14 "gemini-2.5-flash-lite", "gemini-2.5-flash"]`	The list of AI model names to try, newest first. If one model is retired, the next one is tried.
`15 HIST = "history.txt"`	The name of the memory file where past titles are written.
`16 IG_USER = os.environ.get("IG_USER_ID", "").strip()` `17 IG_TOKEN = os.environ.get("IG_TOKEN", "").strip()`	Reads the Instagram account number and the Instagram key from the secret vault. .strip() removes stray spaces.
`18 GRAPH = "https://graph.facebook.com"`	The web address of Meta's service (the company behind Facebook and Instagram).

Part B: Topics and fonts

The list of 10 topics and where the fonts are.

Code (line numbers on the left)	What it means
`20 CATS = [`	CATS is the list of the 10 topics. Each topic stores its English name ('en'), Hindi name ('hi'), short thumbnail badge words in English ('be') and Hindi ('bh'), a color, and 3 backup search words ('fb') used when a search finds nothing.
`21 {"en": "Cars & Automobiles", "hi": "{hindi} {hindi} {hindi}", "be": "CARS", "bh": "{hindi}", "color": "#E53935",` `22 "fb": ["car driving", "car engine", "highway traffic"]},`	Topic 1: Cars and automobiles (red).
`23 {"en": "Space & Rockets", "hi": "{hindi} {hindi} {hindi}", "be": "SPACE", "bh": "{hindi}", "color": "#5C6BC0",` `24 "fb": ["rocket launch", "galaxy stars", "planet earth space"]},`	Topic 2: Space and rockets (indigo).
`25 {"en": "Aviation & Aeroplanes", "hi": "{hindi} {hindi} {hindi}", "be": "AVIATION", "bh": "{hindi}", "color": "#039BE5",` `26 "fb": ["airplane flying", "airport runway", "clouds sky"]},`	Topic 3: Aviation and aeroplanes (sky blue).
`27 {"en": "AI & Machine Learning", "hi": "{hindi} {hindi} {hindi} {hindi}", "be": "AI & ML", "bh": "{hindi}", "color": "#8E24AA",` `28 "fb": ["artificial intelligence", "robot", "data network"]},`	Topic 4: AI and machine learning (purple).
`29 {"en": "Science Explained", "hi": "{hindi} {hindi} {hindi}", "be": "SCIENCE", "bh": "{hindi}", "color": "#43A047",` `30 "fb": ["science laboratory", "microscope", "chemistry"]},`	Topic 5: Science explained (green).
`31 {"en": "Technology & Gadgets", "hi": "{hindi} {hindi} {hindi}", "be": "TECH", "bh": "{hindi}", "color": "#FB8C00",` `32 "fb": ["technology", "circuit board", "smartphone"]},`	Topic 6: Technology and gadgets (orange).
`33 {"en": "Engineering & Machines", "hi": "{hindi} {hindi} {hindi}", "be": "ENGINEERING", "bh": "{hindi}", "color": "#546E7A",` `34 "fb": ["factory machine", "construction site", "bridge"]},`	Topic 7: Engineering and machines (grey-blue).
`35 {"en": "Inventions & History", "hi": "{hindi} {hindi} {hindi}", "be": "INVENTIONS", "bh": "{hindi}", "color": "#D81B60",` `36 "fb": ["old machine", "ancient ruins", "library books"]},`	Topic 8: Inventions and history (pink).
`37 {"en": "Earth & Oceans", "hi": "{hindi} {hindi} {hindi}", "be": "EARTH", "bh": "{hindi}", "color": "#00897B",` `38 "fb": ["ocean waves", "mountains", "volcano"]},`	Topic 9: Earth and oceans (teal).

`39 {"en": "Animals & Nature", "hi": "{hindi} {hindi} {hindi}", "be": "NATURE", "bh": "{hindi}", "color": "#7CB342",`
`40 "fb": ["wild animals", "forest", "birds flying"]},`
`41 ]`
	Topic 10: Animals and nature (light green). The] closes the list.

`43 DEV_FONTS = ["/usr/share/fonts/truetype/noto/NotoSansDevanagari-Bold.ttf"]`	Where the Hindi-capable bold font lives on the GitHub computer.
`44 LAT_FONTS = ["/usr/share/fonts/truetype/noto/NotoSans-Bold.ttf",` `45 "/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf"]`	Where the English bold fonts live. The second one is a backup if the first is missing.

Part C: Small helpers

Three tiny tools used everywhere: retry, run another program, measure a file's length.

Code (line numbers on the left)	What it means
`48 def retry(fn, tries=4, wait=5):`	Defines 'retry', a helper that tries a job again if it fails. fn is the job, tries is how many attempts (default 4) and wait is the pause in seconds (default 5).
`49     last = None`	'last' remembers the most recent error.
`50     for i in range(tries):`	Repeat once for every attempt. i counts 0, 1, 2 and so on.
`51         try:` `52             return fn()`	Try to do the job. If it works, hand back the result and stop.

`53         except Exception as e:`
`54             last = e`
`55             print("retry", i, repr(e)[:300])`
	If it fails: remember the error and print a short message (attempt number and the error) into the log.

`56             time.sleep(wait * (i + 1))`	Wait a little longer each time (wait multiplied by the attempt number) before trying again.
`57     raise RuntimeError(f"failed after retries: {last!r}"[:500])`	If every attempt failed, stop with a clear error that includes the last problem.
`60 def run(cmd):`	Defines 'run', which runs another program (such as ffmpeg) given as a list of words.
`61     r = subprocess.run(cmd, capture_output=True, text=True)`	Runs it and captures its messages as text instead of showing them.

`62     if r.returncode != 0:`
`63         print(r.stderr[-1500:])`
`64         raise RuntimeError("command failed: " + cmd[0])`
	If the program reported a problem, print the last 1500 characters of its message and stop with an error naming the program.

`67 def duration(path):`	Defines 'duration', which finds how long an audio or video file is.

`68     out = subprocess.check_output(["ffprobe", "-v", "error", "-show_entries", "format=duration",`
`69                                    "-of", "default=nw=1:nk=1", path])`
`70     return float(out.strip())`
	Runs ffprobe (a file inspector) to read the duration and converts the answer into a number of seconds.

Part D: Memory and choosing the topic

Reading and writing history.txt and choosing today's topic.

Code (line numbers on the left)	What it means
`73 # ---------- history ----------` `74 def read_history():`	A heading comment, then the definition of read_history, which reads the memory file.
`75     rows, used = [], []`	'rows' will hold the structured records and 'used' will hold every past title.
`76     if os.path.exists(HIST):` `77         for line in open(HIST, encoding="utf-8").read().splitlines():`	If the memory file exists, go through it line by line.

`78             line = line.strip()`
`79             if not line:`
`80                 continue`
	Trim spaces from the line and skip it if it is empty.

`81             p = line.split("|", 2)`	Split the line at the '|' sign into at most 3 parts: kind, topic and title.

`82             if len(p) == 3:`
`83                 rows.append(p)`
`84                 used.append(p[2])`
	If the line has all 3 parts it is a proper record: save the record and remember its title.

Code (line numbers on the left)	What it means
85 else: 86 used.append(line)	Otherwise (older plain lines) just remember the whole line as a used title.
87 return rows, used	Give back both lists.
90 def add_history(kind, cat, title):	Defines add_history, which writes a new record.
91 with open(HIST, "a", encoding="utf-8") as f: 92 f.write(f"{kind}|{cat['en']}|{title.replace('|', '/')}\n")	Opens the memory file in 'append' mode (adds to the end) and writes kind|topic|title plus a new line. A '|' inside the title is swapped for '/' so records never break.
95 def pick_cat(kind, rows):	Defines pick_cat, which chooses today's topic.
96 n = sum(1 for r in rows if r[0] == kind)	n is how many videos of this kind were already made.
97 shift = 0 if kind == "long" else len(CATS) // 2	Shorts are shifted by half the list (5 places) so they never share the topic of the long video.
98 return CATS[(n + shift) % len(CATS)]	Picks topic number n plus the shift. The % sign wraps around to the start of the list after the last topic.

Part E: The AI writer and the voice

Asking Gemini for the plan, checking its answer, and making the voice.

Code (line numbers on the left)	What it means
101 # ---------- content ---------- 102 def gemini(prompt):	A heading comment, then the definition of gemini, which sends instructions to the AI and gets the answer.
103 last = None	'last' remembers the latest error.
104 for m in MODELS: 105 try:	For every model name in the list, try it.
106 r = requests.post(107 f"https://generativelanguage.googleapis.com/v1beta/models/{m}:generateContent?key={GEMINI}", 108 json={"contents": [{"parts": [{"text": prompt}]}], 109 "generationConfig": {"responseMimeType": "application/json", "temperature": 0.8}}, 110 timeout=180)	Sends the instructions (prompt) to Google's AI address for that model, with your key. It asks for the answer in JSON format with a creativity level of 0.8 and waits up to 180 seconds.
111 r.raise_for_status()	If Google answered with an error code, raise an error.
112 text = r.json()["candidates"][0]["content"]["parts"][0]["text"]	Digs the text of the answer out of Google's reply.
113 print("used model", m) 114 return json.loads(text)	Prints which model worked and returns the answer, converted from JSON text into Python data.
115 except Exception as e: 116 print("model failed", m, repr(e)[:200]) 117 last = e	If that model failed: print it, remember the error and move on to the next model.
118 raise last	If every model failed, stop with the last error.
121 def clean(d, kind):	Defines clean, which checks and tidies the AI's answer.
122 need = {"title", "description", "tags", "script", "keywords", "thumb_text"} 123 assert isinstance(d, dict) and need <= set(d), "missing keys"	'need' is the list of 6 pieces the answer must contain. 'assert' stops the program with an error if the answer is not a dictionary or lacks any of them.
124 d["title"] = str(d["title"]).replace("<", "").replace(">", "").strip()[:95] 125 d["description"] = str(d["description"]).replace("<", "").replace(">", "").strip()[:4500]	Title and description: convert to text, delete the < and > signs (YouTube rejects them), trim spaces and cut to the maximum length (95 and 4500 characters).

Code (line numbers on the left)	What it means
`126 d["script"] = " ".join(str(d["script"]).split())`	Script: squash all extra spaces and new lines into single spaces.
`127 d["thumb_text"] = " ".join(str(d["thumb_text"]).split())[:40]`	Thumbnail text: tidy the same way and keep at most 40 characters.
`128 d["keywords"] = [str(k).strip() for k in d["keywords"] if str(k).strip()][:16]`	Keywords: keep only the non-empty ones, at most 16.
`129 tags, total = [], 0`	Prepares an empty tag list and a running character count.

`130 for t in d["tags"]:`
`131 t = str(t).replace("<", "").replace(">", "").strip()`
`132 if t and total + len(t) < 400:`
`133 tags.append(t)`
`134 total += len(t) + 1`
`135 d["tags"] = tags[:15]`
	Goes through the tags: cleans each one and adds it if the total stays under 400 characters (YouTube's limit is 500). Then keeps at most 15.

`136 assert d["title"] and d["description"], "empty title or description"`	Stops if the title or description is empty.
`137 assert len(d["script"].split()) >= (200 if kind == "long" else 50), "script too short"`	Stops if the script is too short (under 200 words for a long video, under 50 for a Short).
`138 assert len(d["keywords"]) >= 4, "too few keywords"`	Stops if there are fewer than 4 keywords.
`139 if kind == "short" and "#shorts" not in d["description"].lower():` `140 d["description"] += "\n\n#Shorts"`	For a Short, adds #Shorts at the end of the description if it is missing. The check ignores capital letters.
`141 return d`	Hands back the cleaned answer.
`144 def get_content(kind, cat, used):`	Defines get_content, which asks the AI for today's complete video plan.

`145 if kind == "long":`
`146 fmt, words, nk = "a 3-4 minute educational explainer video", "450-550", 14`
`147 else:`
`148 fmt, words, nk = "a YouTube Short of about 40 seconds", "90-110", 6`
	Sets the format. Long video: a 3 to 4 minute explainer, 450 to 550 word script, 14 keywords. Short: about 40 seconds, 90 to 110 words, 6 keywords.

`149 thumb_note = " (pure Devanagari, write English words like AI in Devanagari)" if HI else ""`	An extra note for Hindi thumbnails (pure Devanagari letters). It is empty for English.

`150 prompt = f"""You are a professional educational YouTube creator.`
`151 Topic category: {cat['en']}. Language: {LANG_NAME}.`
`152 Format: {fmt}.`
`153 Titles already used (never repeat or closely copy): {' | ' .join(used[-60:])}`
	Starts the instruction message. It tells the AI it is an educational creator and gives the topic, language, format and the last 60 titles to avoid repeating.

`154 Rules: educational and factually accurate; use only well-known, verifiable facts; do not invent statistics, dates or quotes; no medical, financial or political claims; no brand logos or copyrighted material; clear, friendly and engaging; the first sentence is a strong hook; end with a short invitation to subscribe. The script is plain spoken text only: no emojis, no stage directions, no markdown, no brackets.{' Write technical English terms in Devanagari so the voice pronounces them correctly.' if HI else ''}`	The rules: accurate educational facts only, no invented numbers or quotes, no medical, money or political claims, no logos or copyrighted material, friendly tone, a strong first sentence, an invitation to subscribe, and a plain spoken script. For Hindi it also asks for English terms to be written in Devanagari so the voice says them correctly.

`155 Return ONLY JSON with these keys:`
`156 title: max 70 characters, curiosity-driven, honest`
`157 description: 120-200 words, first line is a hook, end with 3 hashtags`
`158 tags: list of 10 short tags`
`159 script: {words} words`
`160 keywords: list of {nk} different English stock-footage search terms (1-3 words each), concrete things that can be filmed, in the order of the script`
`161 thumb_text: 2 to 4 punchy words{thumb_note}"""`
	Tells the AI to reply only as JSON with six items: title, description, tags, script (with the word count), keywords (stock-footage search words) and thumbnail text.

`162 return retry(lambda: clean(gemini(prompt), kind), 4, 8)`	Sends it, checks the answer with clean, and if the AI gave a bad answer tries again (4 times, waiting longer each time).

Code (line numbers on the left)	What it means
165 `def make_voice(text, path, rate):` 166 `    asyncio.run(edge_tts.Communicate(text, VOICE, rate=rate).save(path))`	Defines make_voice and runs edge-tts: it turns the script into a voice file at the requested speed (rate). asyncio.run is needed because edge-tts works in 'async' style.

Part F: Downloading clips and editing the video

Getting Pixabay clips and building the final video with FFmpeg.

Code (line numbers on the left)	What it means
169 `# ---------- clips ----------` 170 `def fetch_clips(keywords, fallback, need, big, prefix):`	A heading comment, then the definition of fetch_clips, which downloads stock videos. Inputs: search words, backup words, how many clips are needed, 'big' (use the larger file for Shorts) and a file-name prefix.
171 `    paths, seen = [], set()`	'paths' are the files downloaded so far and 'seen' are the clip IDs already used, to avoid duplicates.
173 `    def get_one(kw):`	Defines get_one inside it, which gets one clip for one search word.
174 `        r = requests.get("https://pixabay.com/api/videos/",` 175 `                         params={"key": PIXABAY, "q": kw[:90], "per_page": 15, "safesearch": "true"},` 176 `                         timeout=60)`	Asks Pixabay's video search with your key and the search word (max 90 characters), for 15 results, with safe-search on.
177 `        r.raise_for_status()`	Raise an error if Pixabay refuses.
178 `        hits = [h for h in r.json().get("hits", []) if h["id"] not in seen and h.get("duration", 10) >= 4]`	Keeps only results that were not used before and are at least 4 seconds long.
179 `        if not hits:` 180 `            return None`	If nothing is left, return None (no clip found).
181 `        h = random.choice(hits[:5])`	Picks randomly among the top 5 results so videos do not always use the same clip.
182 `        v = h["videos"]`	v holds the available file sizes of that clip.
183 `        order = ["large", "medium", "small", "tiny"] if big else ["medium", "small", "large", "tiny"]`	Order of preference: Shorts want the biggest file first, long videos want the 1080p one (medium) first.
184 `        url = next(v[k]["url"] for k in order if k in v and v[k].get("url"))`	Takes the first size in that order that really has a download link.
185 `        p = f"{prefix}{len(paths)}.mp4"`	The file name, for example long_clip0.mp4.
186 `        with requests.get(url, stream=True, timeout=(30, 300)) as resp:` 187 `            resp.raise_for_status()` 188 `            with open(p, "wb") as f:` 189 `                for chunk in resp.iter_content(1 << 20):` 190 `                    f.write(chunk)`	Downloads the file in 1 MB pieces and saves it (stream=True keeps memory use low). It allows 30 seconds to connect and 300 to read.
191 `        seen.add(h["id"])` 192 `        return p`	Remembers this clip's ID and returns the file name.
194 `    for kw in list(keywords) + list(fallback):` 195 `        if len(paths) >= need:` 196 `            break`	Goes through the search words and then the backup words, and stops as soon as enough clips are collected.
197 `        try:` 198 `            p = retry(lambda: get_one(kw), 3, 4)` 199 `            if p:` 200 `                paths.append(p)`	Tries (up to 3 times) to get a clip. If it worked, adds it to the list.
201 `            else:` 202 `                print("no clip for", kw)`	If no clip matched the word, prints a note.
203 `        except Exception as e:` 204 `            print("skip keyword", kw, repr(e)[:200])`	If the attempts crashed, prints 'skip' and carries on with the next word.

Code (line numbers on the left)	What it means
205 assert len(paths) >= 3, "not enough clips"	Stops with an error if fewer than 3 clips were found.
206 return paths	Returns the list of clip files.
209 def build_video(clips, voice, out, w, h, tag):	Defines build_video, which assembles the clips and the voice into the final video file.
210 dur = duration(voice)	dur is the length of the voice in seconds.
211 per = (dur + 0.8) / len(clips)	per is how long each clip is shown: voice length plus 0.8 seconds spare, divided by the number of clips.
212 vf = (f"scale={w}:{h}:force_original_aspect_ratio=increase,crop={w}:{h}," 213 f"fps=30,setsar=1,format=yuv420p")	The 'video filter' recipe: scale the clip so it fills the frame, crop the extra edges to exactly w by h, use 30 frames per second, square pixels and a color format every player understands.
214 segs = []	An empty list for the cut pieces.
215 for i, c in enumerate(clips): 216 s = f"{tag}_seg{i}.mp4"	For each clip, makes a name for its piece file.
217 run(["ffmpeg", "-y", "-stream_loop", "-1", "-i", c, "-t", f"{per:.3f}", "-vf", vf, "-an", 218 "-c:v", "libx264", "-preset", "veryfast", "-crf", "21", "-r", "30", s])	Runs ffmpeg: loop the clip if it is too short (-stream_loop -1), keep only 'per' seconds, apply the recipe, remove sound (-an), encode as H.264 quickly (veryfast) at quality 21 and 30 frames per second.
219 segs.append(s)	Remembers the piece.
220 lst = f"{tag}_list.txt" 221 with open(lst, "w") as f: 222 f.write("".join(f"file '{s}'\n" for s in segs))	Writes a text list of all the pieces (one 'file ...' line each) for ffmpeg to join.
223 joined = f"{tag}_joined.mp4" 224 run(["ffmpeg", "-y", "-f", "concat", "-safe", "0", "-i", lst, "-c", "copy", joined])	Joins the pieces end to end without re-encoding (-c copy) into one joined file.
225 run(["ffmpeg", "-y", "-i", joined, "-i", voice, "-map", "0:v", "-map", "1:a", 226 "-c:v", "copy", "-c:a", "aac", "-b:a", "192k", "-shortest", "-movflags", "+faststart", out])	Adds the voice: video from the joined file, audio from the voice file. The video is copied, the sound is encoded as AAC, it stops at the shorter input, and file info is moved to the front for quick streaming.
227 for s in segs + [lst, joined]: 228 try: 229 os.remove(s) 230 except OSError: 231 pass	Cleans up by deleting all temporary pieces. It ignores errors if one is already gone.

Part G: Making the thumbnail

Drawing the cover picture with Pillow.

Code (line numbers on the left)	What it means
234 # ---------- thumbnail ---------- 235 def find_font(cands):	A heading comment, then find_font, which looks for a font file.
236 for p in cands: 237 if os.path.exists(p): 238 return p	Checks each candidate path and returns the first one that exists.
239 raise RuntimeError("font not found: " + cands[0])	If none exist, stops with an error.
242 def is_dev(ch): 243 return "\u0900" <= ch <= "\u097F" or ch in "\u200c\u200d"	Defines is_dev: True if a character is Devanagari (the Hindi script), meaning Unicode range U+0900 to U+097F, or an invisible joiner mark.
246 def runs(text): 247 out, cur = [], None	Defines runs, which cuts text into pieces of the same script. 'out' is the result and 'cur' is the current type.

Code (line numbers on the left)	What it means
248 for ch in text: 249 t = "d" if is_dev(ch) else ("l" if ch.isalpha() else None)	For each character decide: 'd' for Devanagari, 'l' for a Latin letter, or None for spaces, numbers and punctuation.
250 if t is None: 251 t = cur or "l"	Spaces and punctuation join the current piece (or Latin at the very start).
252 if t == cur: 253 out[-1][1] += ch	Same type as before: add the character to the last piece.
254 else: 255 out.append([t, ch]) 256 cur = t	A different type: start a new piece.
257 return out	Returns the pieces.
260 def text_w(d, text, fd, fl): 261 return sum(d.textlength(s, font=(fd if t == "d" else fl)) for t, s in runs(text))	Defines text_w, which measures the pixel width of text by adding the width of each piece in its proper font.
264 def draw_text(d, x, y, text, fd, fl, fill, sw): 265 for t, s in runs(text): 266 f = fd if t == "d" else fl 267 d.text((x, y), s, font=f, fill=fill, anchor="ls", stroke_width=sw, stroke_fill="black") 268 x += d.textlength(s, font=f)	Defines draw_text. It draws each piece in its own font with a black outline (stroke). The anchor 'ls' means left-baseline, so mixed fonts line up. Then x moves right by the piece's width.
271 def wrap(d, text, fd, fl, maxw): 272 lines, cur = [], ""	Defines wrap, which splits text into lines no wider than maxw.
273 for w in text.split(): 274 t = (cur + " " + w).strip()	For each word, tries adding it to the current line.
275 if text_w(d, t, fd, fl) > maxw and cur: 276 lines.append(cur) 277 cur = w	If the line gets too wide, saves the finished line and starts a new one with this word.
278 else: 279 cur = t	Otherwise accepts the longer line.
280 lines.append(cur) 281 return lines	Saves the last line and returns all lines.
284 def make_thumb(frame, text, cat, out): 285 W, H = 1280, 720	Defines make_thumb. W and H set the standard YouTube thumbnail size of 1280 by 720.
286 img = Image.open(frame).convert("RGB") 287 img = ImageOps.fit(img, (W, H), Image.Resampling.LANCZOS)	Opens the video frame and crops and resizes it to fill exactly 1280 by 720.
288 img = ImageEnhance.Color(img).enhance(1.25) 289 img = ImageEnhance.Contrast(img).enhance(1.1) 290 img = ImageEnhance.Brightness(img).enhance(0.9)	Makes the picture more vivid: color plus 25 percent, contrast plus 10 percent, slightly darker (90 percent).
291 grad = ImageOps.invert(Image.linear_gradient("L").rotate(90, expand=True).resize((960, H))) 292 grad = grad.point(lambda v: int(v * 0.93)) 293 img.paste(Image.new("RGB", (960, H), (0, 0, 0)), (0, 0), grad)	Makes a black-to-clear fade on the left 960 pixels (a gradient, rotated and flipped, at 93 percent strength) and pastes it on, so the text is easy to read.
294 d = ImageDraw.Draw(img)	Gets a drawing pen for the image.
295 devp, latp = find_font(DEV_FONTS), find_font(LAT_FONTS)	Finds the Hindi font file and the English font file.
296 maxw, x0 = 760, 70 297 text = text.upper() if not HI else text	The text area is at most 760 pixels wide and starts 70 from the left. English headlines are made CAPITAL.
298 size = 190	Starts with huge letters (size 190).
299 while True: 300 fd, fl = ImageFont.truetype(devp, size), ImageFont.truetype(latp, size) 301 lines = wrap(d, text, fd, fl, maxw) 302 fits = all(text_w(d, ln, fd, fl) <= maxw for ln in lines) and len(lines) <= 3 303 lh = int(size * 1.28) 304 if (fits and lh * len(lines) <= 470) or size <= 70: 305 break 306 size -= 6	Keeps shrinking the letters by 6 until the headline fits in at most 3 lines and 470 pixels of height (or the size reaches 70). Each round it loads the fonts, wraps the lines and checks the fit.

Code (line numbers on the left)	What it means
307 top = 380 - (lh * len(lines)) // 2 + int(size * 0.98)	Works out where the first line sits so that the whole text block is centered vertically.
308 for i, ln in enumerate(lines): 309 col = "#FFD400" if i == len(lines) - 1 else "white" 310 draw_text(d, x0, top + i * lh, ln, fd, fl, col, max(6, size // 22))	Draws each line: the last line yellow, the others white. The outline gets thicker as the letters get bigger.
311 badge = cat["bh"] if HI else cat["be"]	Picks the badge word (Hindi or English) of today's topic.
312 bf_d, bf_l = ImageFont.truetype(devp, 44), ImageFont.truetype(latp, 44) 313 bw = int(text_w(d, badge, bf_d, bf_l)) + 56	Loads the badge fonts at 44 pixels and measures the badge width plus 56 pixels of padding.
314 d.rounded_rectangle((x0, 56, x0 + bw, 130), radius=16, fill=cat["color"]) 315 draw_text(d, x0 + 28, 106, badge, bf_d, bf_l, "white", 0)	Draws a rounded colored box at the top-left and writes the badge word in white.
316 d.rectangle((0, H - 16, W, H), fill=cat["color"])	Draws a 16 pixel color bar across the bottom.
317 for q in (92, 85, 78, 70): 318 img.save(out, "JPEG", quality=q) 319 if os.path.getsize(out) < 1_900_000: 320 break	Saves the picture as JPEG, trying quality 92, 85, 78 and 70 until the file is under 1.9 MB (YouTube's limit is 2 MB).

Part H: YouTube

Connecting, choosing the go-live time, uploading and making playlists.

Code (line numbers on the left)	What it means
323 # ---------- YouTube ---------- 324 def yt_client(): 325 info = json.loads(os.environ["YT_TOKEN_JSON"]) 326 return build("youtube", "v3", credentials=Credentials.from_authorized_user_info(info))	A heading comment, then yt_client: reads the YT_TOKEN_JSON secret as data and builds the connection to YouTube using the stored permission.
329 def publish_time():	Defines publish_time, which decides when the videos go live.
330 now = datetime.datetime.now(datetime.timezone.utc)	'now' is the current time in UTC (world standard time).
331 t = now.replace(hour=2, minute=30, second=0, microsecond=0)	t is today's 02:30 UTC, which is exactly 8:00 AM in India.
332 if t - now > datetime.timedelta(minutes=4): 333 return t.strftime("%Y-%m-%dT%H:%M:%SZ")	If 8:00 is more than 4 minutes away, return that time as text for YouTube's scheduler.
334 return None	Otherwise return None, which means 'publish immediately'.
337 def upload(yt, path, c, kind, when):	Defines upload, which sends a video to YouTube.
338 status = {"privacyStatus": "private" if when else "public", "selfDeclaredMadeForKids": False}	The status: private if there is a schedule, otherwise public. It also says 'not made for kids'.
339 if when: 340 status["publishAt"] = when	If scheduled, adds the go-live time.
341 body = {"snippet": {"title": c["title"], "description": c["description"], "tags": c["tags"], 342 "categoryId": "27", "defaultLanguage": "hi" if HI else "en"}, 343 "status": status}	The details: title, description, tags, category 27 (Education), language, plus the status.
344 req = yt.videos().insert(part="snippet,status", body=body, 345 media_body=MediaFileUpload(path, chunksize=-1, resumable=True))	Prepares the upload request with the video file, sent in one go and able to resume if interrupted.
346 resp = None 347 while resp is None: 348 _, resp = req.next_chunk()	Sends the file and repeats until YouTube gives its answer.

Code (line numbers on the left)	What it means
349 print("uploaded", kind, resp["id"], "scheduled for " + when if when else "public now")	Prints which video was uploaded and when it goes live.
350 return resp["id"]	Returns the video's ID.
353 def playlist_id(yt, title): 354 token = None	Defines playlist_id, which finds a playlist by name or creates it. 'token' marks which page of results we are on.
355 while True: 356 r = yt.playlists().list(part="snippet", mine=True, maxResults=50, pageToken=token).execute()	Loops: asks YouTube for up to 50 of your playlists per page.
357 for it in r.get("items", []): 358 if it["snippet"]["title"] == title: 359 return it["id"]	Looks for a playlist with the same title and returns its ID.
360 token = r.get("nextPageToken") 361 if not token: 362 break	Moves to the next page of results. If there are no more pages, stops looking.
363 r = yt.playlists().insert(part="snippet,status", body={ 364 "snippet": {"title": title, "description": title}, 365 "status": {"privacyStatus": "public"}}).execute()	Not found: creates a new public playlist with that title.
366 return r["id"]	Returns the new playlist's ID.
369 def add_to_playlist(yt, title, vid): 370 pid = playlist_id(yt, title)	Defines add_to_playlist and finds (or creates) the playlist.
371 yt.playlistItems().insert(part="snippet", body={"snippet": { 372 "playlistId": pid, "resourceId": {"kind": "youtube#video", "videoId": vid}}}).execute() 373 print("added to playlist", title)	Adds the video to the playlist and prints a message.

Part I: Instagram

Uploading the Reel, waiting for processing and publishing at 8:00.

Code (line numbers on the left)	What it means
376 # ---------- Instagram ---------- 377 def ig_prepare(path, caption):	A heading comment, then ig_prepare, which uploads the Short to Instagram but does not publish it yet.
378 size = os.path.getsize(path)	The file size in bytes.
379 r = requests.post(f"{GRAPH}/{IG_USER}/media", data={ 380 "media_type": "REELS", "upload_type": "resumable", "share_to_feed": "true", 381 "caption": caption, "access_token": IG_TOKEN}, timeout=60)	Step 1: tells Instagram 'I am going to upload a Reel', with the caption and key. It also asks to show the Reel on the profile grid.
382 if r.status_code != 200: 383 raise RuntimeError("IG create failed: " + r.text[:300])	If Instagram refuses, stop with a message.
384 j = r.json() 385 cid, uri = j["id"], j["uri"]	Reads the reply: cid is the draft's ID and uri is the address to upload to.
386 with open(path, "rb") as f: 387 up = requests.post(uri, headers={"Authorization": f"OAuth {IG_TOKEN}", "offset": "0", 388 "file_size": str(size)}, data=f, timeout=900)	Step 2: sends the video file to that address, with the key and the file size in the headers.
389 if up.status_code != 200: 390 raise RuntimeError("IG upload failed: " + up.text[:300])	If the upload failed, stop with a message.
391 for _ in range(60): 392 st = requests.get(f"{GRAPH}/{cid}", params={"fields": "status_code,status", 393 "access_token": IG_TOKEN}, timeout=60).json()	Step 3: up to 60 times, asks Instagram how the processing is going.

Code (line numbers on the left)	What it means
394 code = st.get("status_code") 395 if code == "FINISHED": 396 return cid	If the status is FINISHED, return the draft's ID.
397 if code in ("ERROR", "EXPIRED"): 398 raise RuntimeError("IG processing failed: " + str(st)[:300])	If the status is ERROR or EXPIRED, stop with a message.
399 time.sleep(10)	Wait 10 seconds and ask again.
400 raise RuntimeError("IG processing timeout")	After about 10 minutes of waiting, give up.
403 def ig_publish(cid): 404 r = requests.post(f"{GRAPH}/{IG_USER}/media_publish", 405 data={"creation_id": cid, "access_token": IG_TOKEN}, timeout=120)	Defines ig_publish, which tells Instagram to publish the draft. It sends the draft's ID and the key.
406 if r.status_code != 200: 407 raise RuntimeError("IG publish failed: " + r.text[:300])	If it failed, stop with a message.
408 print("instagram posted", r.json().get("id"))	Prints the ID of the new Instagram post.
411 def wait_until_publish(): 412 when = publish_time()	Defines wait_until_publish and asks publish_time for the 8:00 AM time.
413 if not when: 414 return	If there is no time (we are already late), return at once.
415 target = datetime.datetime.strptime(when, "%Y-%m-%dT%H:%M:%SZ").replace(tzinfo=datetime.timezone.utc)	Converts that text time into a real date-time called 'target'.
416 secs = (target - datetime.datetime.now(datetime.timezone.utc)).total_seconds()	'secs' is the number of seconds left until the target.
417 if 0 < secs < 4 * 3600: 418 print("waiting", int(secs), "seconds for 8:00 AM IST") 419 time.sleep(secs)	If it is between 0 and 4 hours away, print a note and sleep until then.

Part J: Putting it all together

produce makes one video from start to finish, and main runs the whole day.

Code (line numbers on the left)	What it means
422 # ---------- one video ---------- 423 def produce(kind, yt):	A heading comment, then produce, which makes and posts one video of a given kind (long or short).
424 rows, used = read_history()	Reads the history.
425 cat = pick_cat(kind, rows)	Picks today's topic.
426 print("==", kind, "| category:", cat["en"])	Prints the plan in the log.
427 c = get_content(kind, cat, used) 428 print("title:", c["title"])	Asks the AI for the content and prints the title.
429 voice = f"{kind}_voice.mp3"	The voice file's name.
430 for rate in (["+0%"] if kind == "long" else ["+8%", "+18%", "+30%"]): 431 retry(lambda: make_voice(c["script"], voice, rate), 4, 6) 432 if kind == "long" or duration(voice) <= 58: 433 break	Makes the voice. A long video uses normal speed. A Short tries +8 percent, then +18, then +30 until it is 58 seconds or less.
434 big = kind == "short"	'big' is True for Shorts (use bigger source clips).
435 need = len(c["keywords"]) 436 clips = fetch_clips(c["keywords"], cat["fb"], need, big, f"{kind}_clip")	Downloads the clips. The number needed equals the number of keywords.
437 out = f"{kind}.mp4"	The output file's name.
438 w, h = (1920, 1080) if kind == "long" else (1080, 1920) 439 build_video(clips, voice, out, w, h, kind)	Chooses the size (long 1920 by 1080, Short 1080 by 1920) and builds the video.

Code (line numbers on the left)	What it means
440 thumb = None	The thumbnail starts as nothing.
441 if kind == "long": 442 run(["ffmpeg", "-y", "-ss", "1", "-i", clips[0], "-frames:v", "1", "-q:v", "2", "frame.jpg"]) 443 thumb = "thumb.jpg" 444 make_thumb("frame.jpg", c["thumb_text"], cat, thumb)	If it is a long video: takes a still frame at 1 second from the first clip and makes the thumbnail.
445 vid = retry(lambda: upload(yt, out, c, kind, publish_time()), 2, 20)	Uploads the video (2 tries, 20 seconds apart) using the go-live time.
446 add_history(kind, cat, c["title"])	Writes into the history right after the upload so a later problem never repeats the title.
447 if thumb: 448 try: 449 yt.thumbnails().set(videoId=vid, media_body=MediaFileUpload(thumb)).execute() 450 print("thumbnail set") 451 except Exception as e: 452 print("THUMBNAIL FAILED", repr(e)[:300])	If there is a thumbnail, sets it. If that fails it only prints a warning instead of stopping.
453 try: 454 pl = ("{hindi}" if HI else "Daily Shorts") if kind == "short" else (cat["hi"] if HI else cat["en"]) 455 add_to_playlist(yt, pl, vid) 456 except Exception as e: 457 print("PLAYLIST FAILED", repr(e)[:300])	Chooses the playlist name (Shorts playlist for a Short, the topic name for a long video), adds the video and warns if it failed.
458 for p in clips + [voice] + ([] if kind == "short" else [out]): 459 try: 460 os.remove(p) 461 except OSError: 462 pass	Deletes temporary files. The Short's file is kept because Instagram still needs it.
463 if kind == "short": 464 return out, (c["title"] + "\n\n" + c["description"])[:2000] 465 return None	For a Short returns the file and its caption (title plus description, at most 2000 characters). For a long video returns nothing.
468 def main(): 469 for k in ("GEMINI_API_KEY", "PIXABAY_API_KEY", "YT_TOKEN_JSON"): 470 if not os.environ.get(k): 471 sys.exit("missing secret: " + k)	Defines main. It checks that the 3 required secrets exist and, if one is missing, stops and names it.
472 yt = yt_client()	Connects to YouTube.
473 errors, short = [], None	'errors' collects problems and 'short' will hold the Short's result.
474 for kind in ("long", "short"): 475 try: 476 r = produce(kind, yt) 477 if kind == "short": 478 short = r	For the long video and then the Short: make and post it, and keep the Short's result.
479 except Exception as e: 480 print("FAILED", kind, repr(e)[:500]) 481 errors.append(kind)	If one fails, print it and note it, but carry on, so one problem does not stop everything.
482 if short and IG_USER and IG_TOKEN: 483 try: 484 cid = retry(lambda: ig_prepare(short[0], short[1]), 3, 30) 485 wait_until_publish() 486 ig_publish(cid) 487 except Exception as e: 488 print("FAILED instagram", repr(e)[:500]) 489 errors.append("instagram")	If a Short exists and the Instagram keys are set: prepare the Reel, wait for 8:00 and publish. A failure is noted.
490 elif short: 491 print("instagram skipped: IG_USER_ID / IG_TOKEN not set")	If a Short exists but there are no Instagram keys, print a 'skipped' message.
492 if errors: 493 sys.exit("failed: " + ", ".join(errors))	If anything failed, stop with a 'failed' message. That turns the run red and GitHub sends you an email.
496 if __name__ == "__main__": 497 main()	A standard Python line: only run main() when this file is started directly.